

MSX ONLINE

CLIENTE MSX

Documentación técnica del cliente · Fase 3

Plataforma: MSX2 · MSX-DOS · MSXgl · SDCC
Hardware de red: ObsoNET + InterNestor Lite (UNAPI TCP/IP 1.1)
Versión: 1.0 · Archivos: protocol.h · network.h · msxonline.c · Makefile

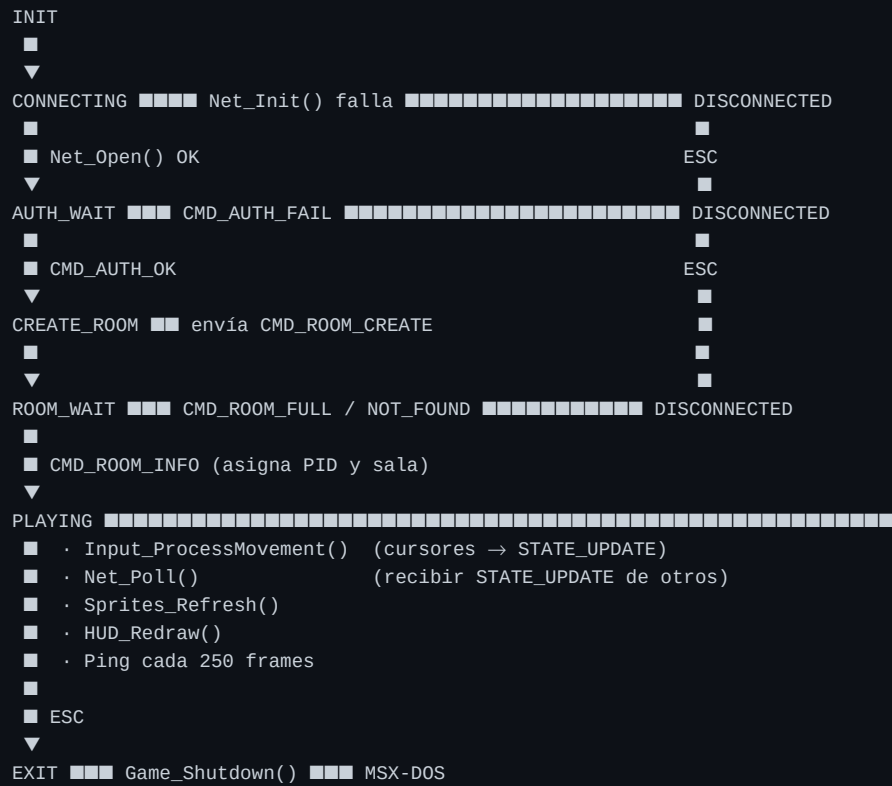
ARCHIVO	CONTENIDO	MODIFICA?
protocol.h	Constantes del protocolo binario compartido con el servidor Node.js	Solo el token y GAME_ID
network.h	Adaptador MSXgl UNAPI TCP. Envuelve las llamadas de red.	Si cambian los nombres de función en tu versión MSXgl
msxonline.c	Lógica del juego completa. Máquina de estados, sprites, HUD, input.	IP del servidor VPS
Makefile	Build con SDCC + MSXgl. Genera msxonline.com para MSX-DOS.	Ruta MSXGL_PATH

Este documento describe la arquitectura, el código fuente y los procedimientos de compilación y despliegue del cliente MSX del sistema MSX Online Game Server. El cliente es un juego mínimo de demostración que conecta al servidor, crea una sala y permite controlar una bola de 16×16 píxeles compartida con otros jugadores en tiempo real.

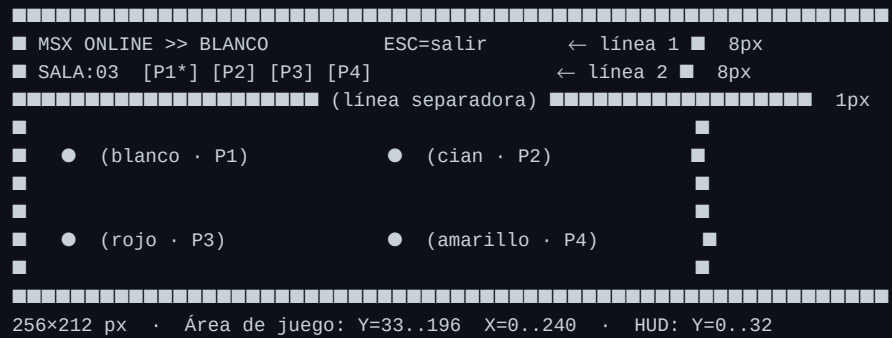
1. VISIÓN GENERAL DEL CLIENTE

El cliente MSX implementa un bucle de juego sincronizado con el VBlank (50/60 Hz) con una máquina de estados finitos que gestiona el ciclo completo de conexión, autenticación, sala y partida.

1.1 Máquina de estados



1.2 Pantalla de juego (Screen 5 — 256×212)



1.3 Colores asignados por PID

PID	JUGADOR	COLOR	ÍNDICE PALETA MSX	POSICIÓN DE INICIO
1	P1 (host)	Blanco	15	X=32, Y=53
2	P2	Cian	7	X=192, Y=53
3	P3	Rojo	9	X=32, Y=153
4	P4	Amarillo	11	X=192, Y=153

■ El color del jugador lo determina su PID asignado por el servidor en CMD_ROOM_INFO. El primer jugador que crea la sala siempre recibe PID=1.

2. PROTOCOL.H — PROTOCOLO BINARIO

Este archivo contiene **únicamente constantes**. No hay código ejecutable. Los mismos valores deben estar presentes en el servidor Node.js. Es la única fuente de verdad del protocolo.

2.1 Estructura del paquete

Byte 0 : 0x46 — Magic 0 ('F' de FX-Media)
Byte 1 : 0x4D — Magic 1 ('M' de FX-Media)
Byte 2 : CMD — Comando (ver tabla 2.2)
Byte 3 : ROOM — ID de sala (0x00 = sin sala)
Byte 4 : PID — Player ID (0x00 = servidor, 0x01..0x04 = jugadores)
Byte 5 : LEN — Longitud del payload en bytes (0..255)
Bytes 6..N : PAYLOAD — Datos del comando (ausente si LEN=0)

Tamaño mínimo: 6 bytes (header sin payload)
Tamaño máximo: 261 bytes (header + 255 bytes payload)

2.2 Tabla de comandos

CMD	HEX	DIRECCIÓN	LEN PAYLOAD	DESCRIPCIÓN
CMD_PING	0x01	MSX→SRV	0	Keepalive. Sin payload.
CMD_PONG	0x02	SRV→MSX	0	Respuesta al ping.
CMD_AUTH	0x10	MSX→SRV	4	Token de autenticación (4 bytes fijos).
CMD_AUTH_OK	0x11	SRV→MSX	0	Autenticación correcta.
CMD_AUTH_FAIL	0x12	SRV→MSX	0	Token incorrecto. Servidor cierra conexión.
CMD_ROOM_CREATE	0x20	MSX→SRV	1	GAME_ID (1 byte). Crea sala nueva.
CMD_ROOM_JOIN	0x21	MSX→SRV	1	ROOM_ID (1 byte). Unirse a sala existente.
CMD_ROOM_LEAVE	0x22	MSX→SRV	0	Abandonar sala actual.
CMD_ROOM_INFO	0x23	SRV→MSX	4	ROOM_ID+GAME_ID+N_PLAYERS+MY_PID.
CMD_ROOM_FULL	0x24	SRV→MSX	0	Sala llena (4/4).
CMD_ROOM_NOT_FOUND	0x25	SRV→MSX	0	Sala solicitada no existe.
CMD_PLAYER_JOINED	0x30	SRV→MSX	1	PID del nuevo jugador.
CMD_PLAYER_LEFT	0x31	SRV→MSX	1	PID del jugador que se fue.
CMD_GAME_START	0x32	SRV→MSX	0	P1 (host) ha iniciado la partida.
CMD_GAME_END	0x33	SRV→MSX	0	Partida terminada.
CMD_STATE_UPDATE	0x40	Ambos	8	Posición/estado de un jugador (relay).
CMD_ERROR	0xFF	Ambos	1	Error genérico (código en payload).

2.3 Payload de CMD_STATE_UPDATE (8 bytes)

Byte 0 : X_HI — Coordenada X, byte alto (X = X_HI<<8 | X_LO)
Byte 1 : X_LO — Coordenada X, byte bajo Rango: 0..240
Byte 2 : Y_HI — Coordenada Y, byte alto (Y = Y_HI<<8 | Y_LO)
Byte 3 : Y_LO — Coordenada Y, byte bajo Rango: 33..196
Byte 4 : FRAME — Frame de animación actual (0..7)
Byte 5 : FLAGS — Bits de estado (ver tabla)
Byte 6 : DATA_0 — Dato extra 0 (reservado, enviar 0)
Byte 7 : DATA_1 — Dato extra 1 (reservado, enviar 0)

BIT	MÁSCARA	NOMBRE	DESCRIPCIÓN
0	0x01	STATE_FLAG_DIR_R	Moviéndose / mirando a la derecha
1	0x02	STATE_FLAG_DIR_D	Moviéndose / mirando hacia abajo
2	0x04	STATE_FLAG_ACTION_A	Botón A pulsado
3	0x08	STATE_FLAG_ACTION_B	Botón B pulsado
4–7	—	—	Reservados para uso futuro

2.4 Token de autenticación

■ El token por defecto (0xDE 0xAD 0xBE 0xEF) es solo para desarrollo. Cambiar en protocol.h antes de desplegar en producción. Debe coincidir exactamente con AUTH_TOKEN en msx-gameserver.js.

3. NETWORK.H — ADAPTADOR MSXGL UNAPI TCP

Esta capa de adaptación **aisla completamente el resto del juego de las llamadas a MSXgl**. Si los nombres de función de tu versión de MSXgl difieren de los usados aquí, solo debes modificar este archivo. El resto del código (msxonline.c) no cambia.

■ Cada función wrapper está marcada con el comentario "■ ADAPTAR ■" en el código fuente.

3.1 API del adaptador

FUNCIÓN	DEVUELVE	PARÁMETROS	DESCRIPCIÓN
Net_Init()	u8 NET_OK/NET_ERROR-		Busca implementación UNAPI TCP/IP. Llamar una vez al inicio.
Net_Open()	NetConn (u8)	const char* ip, u16 port	Abre conexión TCP al servidor. NET_INVALID_CONN (0xFF) si falla.
Net_Close()	void	NetConn conn	Cierra la conexión TCP.
Net_IsConnected()	bool8	NetConn conn	TRUE si la conexión sigue activa.
Net_Send()	u8 NET_OK/NET_ERROR-	NetConn, const u8*, u16 len	Envía datos por TCP.
Net_Available()	u16	NetConn conn	Bytes disponibles para leer (no bloquea).
Net_Recv()	u16	NetConn, u8* buf, u16 maxLen	Lee hasta maxLen bytes. Solo llamar si Net_Available()>=maxLen.

3.2 Funciones MSXgl a verificar

Antes de compilar, verificar en **engine/src/network/unapi_tcp.h** de tu instalación MSXgl que los siguientes nombres de función son correctos:

FUNCIÓN EN network.h	FUNCIÓN MSXGL USADA	VERIFICAR EN
Net_Init()	UNAPI_FindImplementation("TCP/IP")	unapi.h / unapi_tcp.h
Net_Open()	TCP_Open(ip, port, 0, &conn)	unapi_tcp.h ~L193
Net_Close()	TCP_Close(conn)	unapi_tcp.h
Net_IsConnected()	TCP_IsConnected(conn)	unapi_tcp.h
Net_Send()	TCP_SendData(conn, data, length)	unapi_tcp.h
Net_Available()	TCP_GetIncomingDataSize(conn, &sz)	unapi_tcp.h
Net_Recv()	TCP_ReadData(conn, buffer, maxLen)	unapi_tcp.h

3.3 Patrón de uso correcto en Net_Poll()

```
// Orden obligatorio para no bloquear el Z80:

u16 avail = Net_Available(conn);      // 1. ¿Cuántos bytes hay?
if(avail < PROTO_HEADER_SZ) return;   // 2. Salir si no hay header completo
Net_Recv(conn, header, 6);            // 3. Leer header
payloadLen = header[5];
if(payloadLen > 0) {
    avail = Net_Available(conn);      // 4. Verificar de nuevo para el payload
    if(avail < payloadLen) return;     // 5. Paquete incompleto → esperar
    Net_Recv(conn, payload, payloadLen);
}
// 6. Procesar paquete completo
```

■ NUNCA llamar `Net_Recv()` sin comprobar antes `Net_Available()`. InterNestor procesa TCP en el interrupt de 50 Hz: el payload de un paquete puede no haber llegado todavía cuando llega el header.

4. MSXONLINE.C — ESTRUCTURA DEL CÓDIGO

El archivo msxonline.c contiene toda la lógica del juego organizada en módulos funcionales. No usa ficheros .h adicionales — todo en un único .c para simplificar el proceso de compilación con SDCC.

4.1 Organización interna

SECCIÓN	LÍNEAS APROX.	CONTENIDO
Configuración	~30	Defines: IP, puerto, velocidad, límites de área de juego.
Tipos y enums	~25	GameState (enum), Player (struct).
Constantes de color	~10	g_Colors[], g_ColorNames[] indexados por PID.
Patrón sprite bola	~15	g_BallPattern[32]: círculo 16×16 en formato MSXgl.
Variables globales	~20	Estado de la partida, buffers de red, timers.
Protocolo	~25	Packet_Build() — construye paquetes en g_SendBuf.
Red: envíos	~55	Net_SendAuth/CreateRoom/Leave/Ping/State.
Red: recepción	~80	Net_HandlePacket(), Net_Poll().
Red: conexión	~25	Net_Connect() — inicialización secuencial.
Input/movimiento	~35	Input_ProcessMovement() — cursores + envío de estado.
VDP / Sprites	~40	Game_Init(), Sprites_Refresh().
HUD	~50	HUD_Redraw(), HUD_SetStatus().
Bucle principal	~55	Game_Loop() — VBlank sync + máquina de estados.
Entry point	~5	main() — Init + Loop + Shutdown.

4.2 Tipo GameState

```
typedef enum {
    STATE_INIT           = 0, // Inicial antes de arrancar
    STATE_CONNECTING,     // Llamando Net_Open()
    STATE_AUTH_WAIT,     // Esperando CMD_AUTH_OK del servidor
    STATE_CREATE_ROOM,   // Enviando CMD_ROOM_CREATE
    STATE_ROOM_WAIT,     // Esperando CMD_ROOM_INFO
    STATE_PLAYING,       // Partida activa – bucle de juego
    STATE_LEAVING,       // Enviando CMD_ROOM_LEAVE (no bloqueante)
    STATE_DISCONNECTED,  // Error o conexión perdida
    STATE_EXIT           // Terminar y volver a MSX-DOS
} GameState;
```

4.3 Tipo Player

```
typedef struct {
    u16  x;           // Coordenada X en pantalla (0..240)
    u16  y;           // Coordenada Y en pantalla (33..196)
    u8   frame;       // Frame de animación actual (0..7)
    u8   flags;       // Bits STATE_FLAG_* (dirección, botones)
    bool8 active;     // TRUE si este PID está en la sala
} Player;

// Uso: g_Players[pid] donde pid = 1..4
// g_Players[0] no se usa (PID=0 es reservado para el servidor)
static Player g_Players[5];
```

5. FUNCIONES CLAVE — DESCRIPCIÓN DETALLADA

5.1 Game_Init()

```
// Inicialización completa del sistema gráfico y de red

VDP_SetMode(VDP_MODE_SCREEN5);           // 256×212 · 16 colores · bitmap
VDP_SetBorderColor(0);
VDP_FillVRAM(0x00, 0x0000, 0x1FFFF);    // Limpiar VRAM completa
VDP_SetSprite(VDP_SPRITE_SIZE_16 |      // Sprites 16×16 · sin zoom
              VDP_SPRITE_SCALE_1);

// Cargar el mismo patrón en los 4 slots de sprite
// (el color diferencia a cada jugador en VDP_SetSpriteAttribute)
for(i = 0; i < MAX_PLAYERS; i++)
    VDP_LoadSpritePattern(g_BallPattern, i*4, 4);

Print_SetFont(g_Font_MsxDos);             // Fuente para HUD en Screen 5
Print_SetFontType(PRINT_FONT_1);
```

5.2 Net_Connect()

```
void Net_Connect(void)
{
    // Paso 1: Verificar que InterNestor / UNAPI TCP está disponible
    if(Net_Init() != NET_OK) { g_State = STATE_DISCONNECTED; return; }

    // Paso 2: Abrir conexión TCP al VPS (bloqueante hasta timeout)
    g_Conn = Net_Open(SERVER_IP, SERVER_PORT);
    if(g_Conn == NET_INVALID_CONN) { g_State = STATE_DISCONNECTED; return; }

    // Paso 3: Conexión OK → enviar autenticación inmediatamente
    g_State = STATE_AUTH_WAIT;
    Net_SendAuth();    // Envía [0x46][0x4D][0x10][0][0][4][DE][AD][BE][EF]
}


```

5.3 Net_Poll() — El corazón del sistema de recepción

```
void Net_Poll(void)
{
    u8 maxPkts = 4; // Máx. 4 paquetes por frame — evita bloquear el Z80

    if(!Net_IsConnected(g_Conn)) {
        g_State = STATE_DISCONNECTED; return;
    }

    while(maxPkts--) {
        // 1. ¿Hay un header completo?
        if(Net_Available(g_Conn) < PROTO_HEADER_SZ) break;

        // 2. Leer y validar magic bytes
        Net_Recv(g_Conn, header, PROTO_HEADER_SZ);
        if(header[0]!=0x46 || header[1]!=0x4D) break; // Basura

        // 3. ¿Tenemos el payload?
        payloadLen = header[5];
        if(payloadLen > 0) {
            if(Net_Available(g_Conn) < payloadLen) break; // Incompleto
            Net_Recv(g_Conn, payload, payloadLen);
        }

        // 4. Despachar
        Net_HandlePacket(header[2], header[3], header[4],
                        payload, payloadLen);
    }

    // Ping periódico (cada 250 frames ≈ 5 segundos)
    if(++g_PingTimer >= PING_INTERVAL) { g_PingTimer=0; Net_SendPing(); }
}
```


5.4 Net_HandlePacket() — Procesado de paquetes recibidos

```
CMD_AUTH_OK      → g_State = STATE_CREATE_ROOM
CMD_AUTH_FAIL    → g_State = STATE_DISCONNECTED

CMD_ROOM_INFO    → payload[0]=ROOM_ID · payload[2]=N_PLAYERS · payload[3]=MI_PID
                  Activa g_Players[myPid], asigna posición de inicio
                  g_State = STATE_PLAYING

CMD_ROOM_FULL    → g_State = STATE_DISCONNECTED
CMD_ROOM_NOT_FOUND → g_State = STATE_DISCONNECTED

CMD_PLAYER_JOINED → payload[0]=newPid
                  g_Players[newPid].active = TRUE
                  Posición de inicio según PID

CMD_PLAYER_LEFT  → payload[0]=pid
                  g_Players[pid].active = FALSE

CMD_STATE_UPDATE → Solo si pid != g_MyPid:
                  Actualiza g_Players[pid].x/y/frame/flags
                  (No se redibuja aquí – lo hace Sprites_Refresh en el bucle)

CMD_PONG         → (ignorado – solo verifica que la conexión vive)
```

5.5 Input_ProcessMovement()

```
// Lee joystick 1 (cursores del MSX como joystick virtual)
u8 keys = Joystick_Read(JOY_1);

if(keys & JOY_UP)    me->y -= MOVE_SPEED; // clampado a PLAY_Y0
if(keys & JOY_DOWN)  me->y += MOVE_SPEED; // clampado a PLAY_Y1
if(keys & JOY_LEFT)  me->x -= MOVE_SPEED; // clampado a PLAY_X0
if(keys & JOY_RIGHT) me->x += MOVE_SPEED; // clampado a PLAY_X1

// Solo envía STATE_UPDATE cuando hay movimiento real
// (ahorra ancho de banda con ObsoNET)
if(moved) {
    me->frame = (me->frame + 1) & 0x07; // Animación: 8 frames en loop
    Net_SendState();
}
```

■ La condición "solo si hay movimiento" es importante con ObsoNET: reduce el tráfico al mínimo necesario y da respiro al timer de InterNestor.

5.6 Sprites_Refresh()

```
for(i = 1; i <= MAX_PLAYERS; i++) {
    if(g_Players[i].active) {
        VDP_SetSpriteAttribute(
            i - 1,                // Slot sprite: 0..3
            (u8)(g_Players[i].x & 0xFF), // X
            (u8)(g_Players[i].y & 0xFF), // Y
            (i-1)*4,              // Patrón: bloques 0,4,8,12
            g_Colors[i]           // Color: blanco/cian/rojo/amarillo
        );
    } else {
        // Ocultar sprite: Y=209 (fuera del área visible en MSX)
        VDP_SetSpriteAttribute(i-1, 0, 209, 0, 0);
    }
}
```


7. COMPILACIÓN Y DESPLIEGUE

7.1 Requisitos

HERRAMIENTA	VERSIÓN MÍNIMA	PROPÓSITO	OBTENER EN
SDCC	4.x	Compilador C para Z80	sdcc.sourceforge.net
MSXgl	latest	Librería MSX	github.com/aoineko-fr/MSXgl
hex2bin	any	Convierte .ihx → .com	hex2bin.sourceforge.net
make	any	Sistema de build	GNU make
InterNestor	2.x	Stack TCP/IP UNAPI (MSX)	konamiman.com/msx
RAMHELPR.COM	1.2	Helper UNAPI RAM (MSX)	konamiman.com/msx

7.2 Pasos de compilación

```
# 1. Clonar MSXgl si no lo tienes
git clone https://github.com/aoineko-fr/MSXgl ~/msxgl

# 2. Editar msxonline.c con la IP real del servidor
#   #define SERVER_IP    "1.2.3.4"

# 3. Compilar
MSXGL_PATH=~/msxgl make

# Resultado: msxonline.com (~4-8 KB estimado)
```

7.3 Despliegue en MSX

```
# Copiar a disco MSX-DOS (imagen de disco o hardware real)
# Arrancar el MSX con MSX-DOS 2 o Nextor

A> RAMHELPR.COM           ; Instalar helper UNAPI RAM
A> INTERNESTOR.COM        ; Cargar stack TCP/IP
A> MSXONLINE.COM          ; Lanzar el juego
```

■ *InterNestor Lite 2.x requiere mapper support routines. Con MSX-DOS 2 o Nextor están integradas. Si usas MSX-DOS 1, ejecutar MSR.COM antes que RAMHELPR.COM.*

7.4 Parámetros configurables antes de compilar

DEFINE	ARCHIVO	VALOR POR DEFECTO	DESCRIPCIÓN
SERVER_IP	msxonline.c	"1.2.3.4"	IP del VPS. Cambiar obligatoriamente.
SERVER_PORT	msxonline.c	9876	Puerto TCP del servidor.
MOVE_SPEED	msxonline.c	2	Píxeles por frame. Reducir en MSX lentos.
PING_INTERVAL	msxonline.c	250	Frames entre pings (~5s a 50Hz). Aumentar si la red va bien.
AUTH_TOKEN_0..3	protocol.h	0xDE 0xAD 0xBE 0xEF	Token de auth. CAMBIAR en producción.
GAME_ID_BALL	protocol.h	0x01	ID de juego para este cliente.
MSXGL_PATH	Makefile	\$(HOME)/msxgl	Ruta a la instalación de MSXgl.

8. HUD — INTERFAZ DE USUARIO

El HUD ocupa las 32 primeras líneas de la pantalla (Y=0..32) en Screen 5 (bitmap). Se dibuja solo cuando **g_HUDDirty = TRUE** para no desperdiciar ciclos de CPU en cada frame.

8.1 Estructura visual

Y=0..7 (8px): Línea 1 – "MSX ONLINE >> [estado]"
Color título: Amarillo (índice 11)
Color ">>" : Gris (14)
Color estado: Blanco (15)

Y=8..15 (8px): Línea 2 – Información de sala y jugadores
Sin sala : "Sin sala" en rojo apagado (8)
Con sala : "SALA:XX [P1*] [P2] [P3]..."
Cada [Px] en el color de ese jugador
El asterisco (*) marca al jugador local
Al final : " ESC=salir" en gris (14)

Y=16..32 (1px): Línea separadora en el color del jugador local (g_Colors[g_MyPid])
Y=33..211 : Área de juego (sprites)

8.2 Mensajes de estado (línea 1)

ESTADO DEL JUEGO	MENSAJE MOSTRADO
STATE_CONNECTING / buscando UNAPI	"Conectando..."
UNAPI no encontrado	"UNAPI no hallado"
Net_Open() falla	"Fallo conexion"
Esperando AUTH_OK	"Auth enviada..."
CMD_AUTH_FAIL recibido	"AUTH FALLIDA!"
Creando sala	"Creando sala..."
CMD_ROOM_INFO recibido (jugando)	Nombre de color del jugador ej. "BLANCO"
CMD_ROOM_FULL	"SALA LLENA"
CMD_ROOM_NOT_FOUND	"SALA NO EXISTE"
CMD_GAME_START recibido	"JUEGO INICIADO"
Conexión perdida (IsConnected)	"CONEXION PERDIDA"

9. LIMITACIONES CONOCIDAS Y NOTAS DE IMPLEMENTACIÓN

9.1 Limitaciones del cliente v1.0

- **Sin JOIN a sala existente:** el cliente siempre crea una sala nueva (CMD_ROOM_CREATE). El servidor ya soporta CMD_ROOM_JOIN, pero la UI de selección de sala no está implementada en este cliente mínimo.
- **Sin lobby:** no hay pantalla de listado de salas disponibles.
- **Un solo juego:** el payload de STATE_UPDATE está fijo a 8 bytes. Para otros juegos con más datos habrá que versionar el GAME_ID.
- **P1 siempre crea:** no hay opción de unirse a sala de otro jugador desde este cliente. Funciona bien para pruebas con 2+ jugadores en instancias separadas.
- **Sin confirmación de movimiento:** los movimientos no se validan en el servidor. Esto es intencional — el servidor es un relay puro.

9.2 Comportamiento ante pérdida de conexión

Caso 1: ObsoNET pierde la conexión de red física

- Net_IsConnected() devuelve FALSE en el siguiente Net_Poll()
- g_State = STATE_DISCONNECTED
- HUD muestra "CONEXION PERDIDA"
- ESC sale al MSX-DOS

Caso 2: Servidor cae o reinicia

- Net_IsConnected() puede tardar unos segundos en detectarlo
- Mientras tanto, el ping periódico falla en silencio
- Cuando InterNestor detecta el drop, Net_IsConnected() = FALSE
- Idem caso 1

Caso 3: Timeout de 90s sin datos (configurado en el servidor)

- El servidor cierra la conexión por timeout
- Equivalente al caso 2 desde el punto de vista del cliente

9.3 Rendimiento esperado

MÉTRICA	VALOR ESTIMADO	NOTAS
Tasa de frames	50 Hz	Sincronizado con VBlank MSX2.
Paquetes enviados/s	0..50	Solo al mover (máx 1/frame si se mueve siempre).
Tamaño paquete envío	14 bytes	6 header + 8 payload STATE_UPDATE.
Paquetes recibidos/s	0..50 × N_players	N_players - 1 jugadores adicionales.
Bytes/s estimados	< 2 KB/s	Con 4 jugadores moviéndose sin parar.
Ping keepalive	Cada 5s	Configurable con PING_INTERVAL.
Latencia efectiva	20 ms + RTT	20ms = resolución timer InterNestor (50Hz).

9.4 Próximas mejoras sugeridas (Fase 4+)

- Pantalla de lobby: listar salas disponibles con CMD_ROOM_LIST (pendiente en servidor).
- Opción de unirse a sala existente: introducir ROOM_ID con teclado.
- Efecto visual en desconexión de jugador: parpadeo del sprite antes de desaparecer.
- Animación de la bola: rotar el patrón según la dirección del movimiento.
- Nombre de jugador: campo de 4 caracteres en el payload de STATE_UPDATE (bytes 6–7).